

```

from collections import deque
import random
import numpy as np
try:
    import gym
except ModuleNotFoundError:
    import gymnasium as gym # pip install "gymnasium[classic-control]"

class ReplayBuffer:
    def __init__(self, buffer_size, batch_size):
        self.buffer = deque(maxlen=buffer_size)
        self.batch_size = batch_size

    def add(self, state, action, reward, next_state, done):
        data = (state, action, reward, next_state, done)
        self.buffer.append(data)

    def __len__(self):
        return len(self.buffer)

    def get_batch(self):
        data = random.sample(self.buffer, self.batch_size)

        state = np.stack([x[0] for x in data])
        action = np.array([x[1] for x in data])
        reward = np.array([x[2] for x in data])
        next_state = np.stack([x[3] for x in data])
        done = np.array([x[4] for x in data]).astype(np.int32)

        return state, action, reward, next_state, done

env = gym.make('CartPole-v0', render_mode='human')
replay_buffer = ReplayBuffer(buffer_size=10000, batch_size=32)

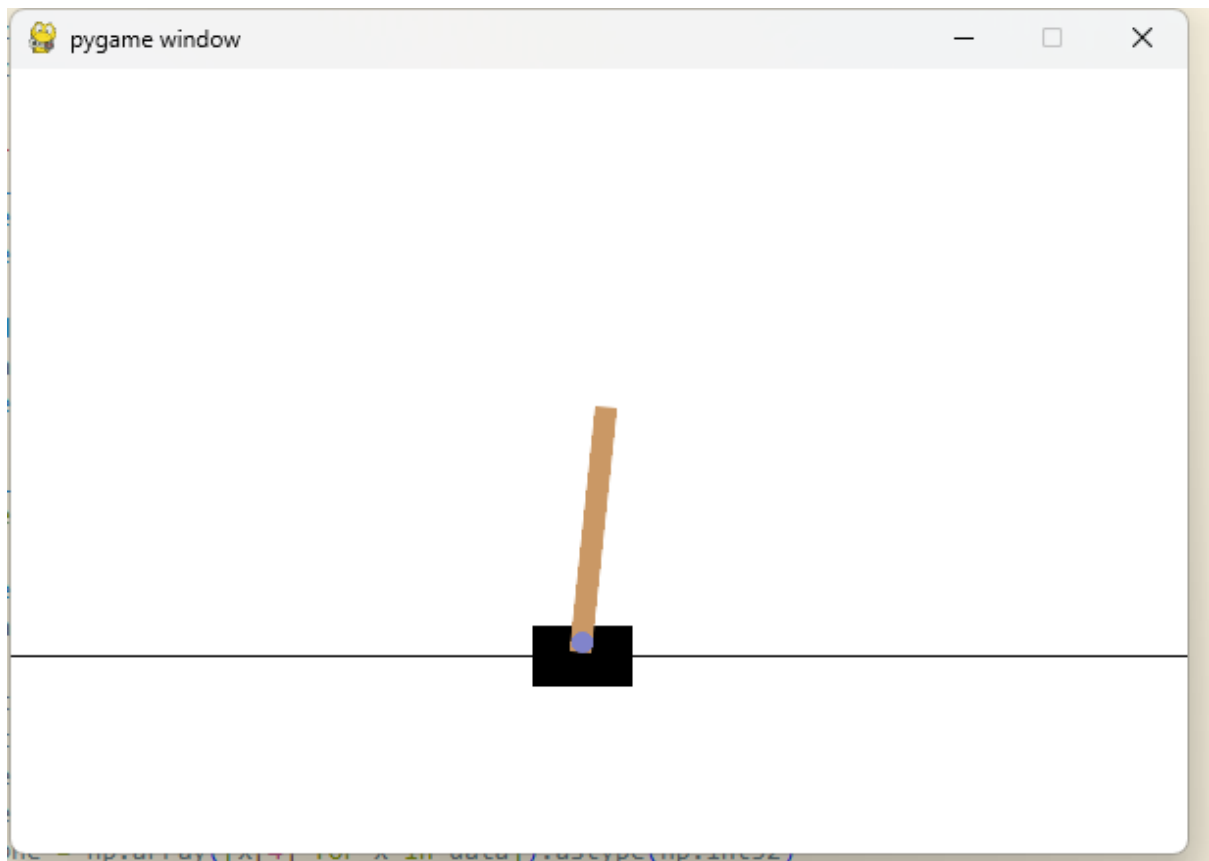
for episode in range(10): # 에피소드 10 회 수행
    state = env.reset()[0]
    done = False

    while not done:
        action = 0 # 항상 0 번째 행동만 수행
        next_state, reward, terminated, truncated, info = env.step(action) #
        경험 데이터 획득
        done = terminated | truncated

        replay_buffer.add(state, action, reward, next_state, done) # 버퍼에
        추가
        state = next_state

```

```
# 경험 데이터 버퍼로부터 미니배치 생성
state, action, reward, next_state, done = replay_buffer.get_batch()
print(state.shape)      # (32, 4)
print(action.shape)     # (32,)
print(reward.shape)     # (32,)
print(next_state.shape) # (32, 4)
print(done.shape)       # (32,)
```



```

import copy
from collections import deque
import random
import matplotlib.pyplot as plt
import numpy as np
try:
    import gym
except ModuleNotFoundError:
    import gymnasium as gym # pip install "gymnasium[classic-control]"
from dezero import Model
from dezero import optimizers
import dezero.functions as F
import dezero.layers as L

class ReplayBuffer:
    def __init__(self, buffer_size, batch_size):
        self.buffer = deque(maxlen=buffer_size)
        self.batch_size = batch_size

    def add(self, state, action, reward, next_state, done):
        data = (state, action, reward, next_state, done)
        self.buffer.append(data)

    def __len__(self):
        return len(self.buffer)

    def get_batch(self):
        data = random.sample(self.buffer, self.batch_size)

        state = np.stack([x[0] for x in data])
        action = np.array([x[1] for x in data])
        reward = np.array([x[2] for x in data])
        next_state = np.stack([x[3] for x in data])
        done = np.array([x[4] for x in data]).astype(np.int32)

        return state, action, reward, next_state, done

class QNet(Model): # ① 신경망 클래스
    def __init__(self, action_size):
        super().__init__()
        self.l1 = L.Linear(128)
        self.l2 = L.Linear(128)
        self.l3 = L.Linear(action_size)

    def forward(self, x):
        x = F.relu(self.l1(x))
        x = F.relu(self.l2(x))

```

```

    x = self.l3(x)
    return x

class DQNAgent: # 에이전트 클래스
    def __init__(self):
        self.gamma = 0.98
        self.lr = 0.0005
        self.epsilon = 0.1
        self.buffer_size = 10000 # 경험 재생 버퍼 크기
        self.batch_size = 32 # 미니배치 크기
        self.action_size = 2

        self.replay_buffer = ReplayBuffer(self.buffer_size, self.batch_size)
        self.qnet = QNet(self.action_size) # ① 원본 신경망
        self.qnet_target = QNet(self.action_size) # ② 목표 신경망
        self.optimizer = optimizers.Adam(self.lr)
        self.optimizer.setup(self.qnet) # ③ 옵티마이저에 qnet 등록
        # 가중치 업데이트는 qnet 만

    def sync_qnet(self): # ④ 두 신경망 동기화
        self.qnet_target = copy.deepcopy(self.qnet) # 깊은복사

    def get_action(self, state):
        if np.random.rand() < self.epsilon:
            return np.random.choice(self.action_size)
        else:
            state = state[np.newaxis, :] # 배치 처리용 차원 추가
            qs = self.qnet(state)
            return qs.data.argmax()

    def update(self, state, action, reward, next_state, done):
        # ① 경험 재생 버퍼에 경험 데이터 추가
        self.replay_buffer.add(state, action, reward, next_state, done)
        if len(self.replay_buffer) < self.batch_size:
            return # 미니배치 크기만큼 쌓이지 않았다면 종료

        # ② 미니배치 생성
        state, action, reward, next_state, done =
self.replay_buffer.get_batch()

        qs = self.qnet(state)
        q = qs[np.arange(self.batch_size), action]

        next_qs = self.qnet_target(next_state)
        next_q = next_qs.max(axis=1)
        next_q.unchain()
        target = reward + (1 - done) * self.gamma * next_q

```

```

    loss = F.mean_squared_error(q, target)

    self.qnet.cleargrads()
    loss.backward()
    self.optimizer.update()

episodes = 300          # 에피소드 수
sync_interval = 20      # 신경망 동기화 주기(20 번째 에피소드마다 동기화)
env = gym.make('CartPole-v0', render_mode='rgb_array')
agent = DQNAgent()
reward_history = []     # 에피소드별 보상 기록

for episode in range(episodes):
    state = env.reset()[0]
    done = False
    total_reward = 0

    while not done:
        action = agent.get_action(state) # epsilon greedy
        next_state, reward, terminated, truncated, info = env.step(action)
        done = terminated | truncated

        agent.update(state, action, reward, next_state, done)
        state = next_state
        total_reward += reward

    if episode % sync_interval == 0:
        agent.sync_qnet() # Target network 동기화

    reward_history.append(total_reward)
    if episode % 10 == 0:
        print("episode : {}, total reward : {}".format(episode, total_reward))

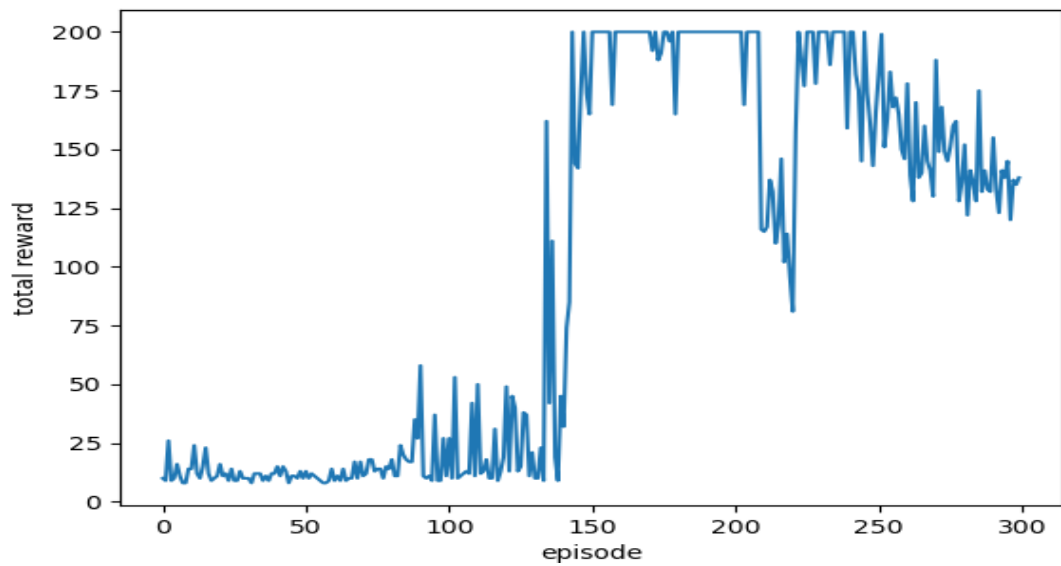
# 카트 폴 에서 에피소드별 보상 총합의 추이
plt.xlabel('episode')
plt.ylabel('total reward')
plt.plot(range(len(reward_history)), reward_history)
plt.show()

# 학습이 끝난 에이전트에 탐욕 행동을 선택하도록 하여 플레이
env2 = gym.make('CartPole-v0', render_mode='human')

agent.epsilon = 0 # 탐욕 정책(무작위로 행동할 확률  $\epsilon$  을 0 으로 설정)
state = env2.reset()[0]
done = False
total_reward = 0

```

```
while not done:
    action = agent.get_action(state)
    next_state, reward, terminated, truncated, info = env2.step(action)
    done = terminated | truncated
    state = next_state
    total_reward += reward
    env2.render()
print('Total Reward:', total_reward)
```



pygame window

